

AI模型智能路由实验报告

课程：智能体开发实战

选题：第25题 AI模型智能路由

姓名：待填写

学号：待填写

班级：待填写

组员：待填写

日期：2026年9月17日

1 需求分析与实现范围

智能体会面对简短问答、材料摘要、代码分析等不同任务。始终调用高成本模型可能造成开销，统一使用轻量模型又可能难以处理复杂问题。本项目实现 RouteLab，根据任务类型、难度和长度选型，输出可解释依据、真实回答及执行统计。

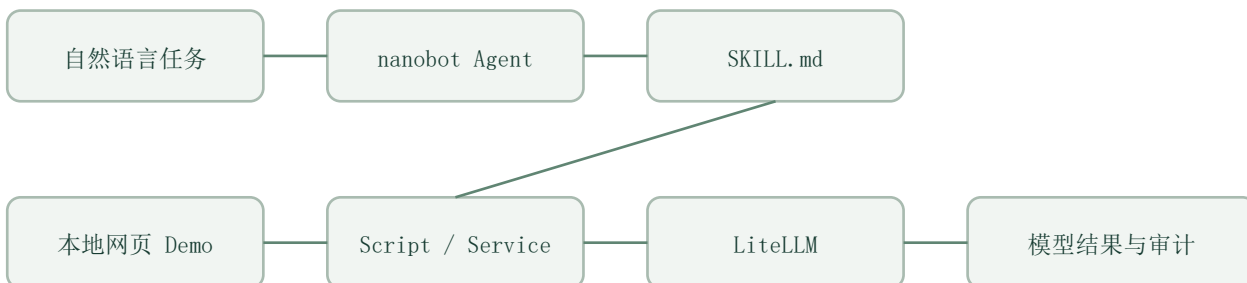
项目通过 LiteLLM 1.101.0 接入 DeepSeek 真实服务，使用 Flash 与 Pro 两个模型，配置快速、均衡、推理三个档位。模型生成需要有效密钥、网络和账户余额。网页、独立CLI和nanobot复用同一个路由核心。

用户可在网页动态配置API密钥。Windows使用当前用户加密保存，密钥不进入仓库、不回显到页面；修改后下次请求立即生效。连接测试读取官方模型列表，已确认账户可访问deepseek-flash和deepseek-v4-pro。

功能目标	可观察结果
三类核心意图	问答、摘要、代码与复杂推理
智能选型	类型、难度、长度、偏好与上下文约束
执行和统计	真实答案、服务返回Token、耗时、费用估算
统一工具接口	Skill说明、独立JSON脚本、Runtime加载
安全与异常	动态加密配置、输入校验、未知用量标识

2 总体架构与开源集成

自然语言经nanobot读取SKILL.md，再调用router.py；脚本进入RouterService，按policy.py选型，由providers.py调用LiteLLM completion，返回结构化结果。网页经本地HTTP接口调用同一个服务。



网页与 CLI 共用核心服务；Agent 通过 Skill 调用独立 Script

config/models.json存放公开模型参数和费率；settings.py负责密钥保存、读取与连接测试；policy.py做校验与规则路由；providers.py是唯一生成边界；service.py管理缓存、计量与SQLite审计；cli.py提供JSON协议。

LiteLLM实际传递model、messages、api_base、api_key、max_tokens和thinking。快速、均衡档显式关闭思考；推理档开启思考，reasoning_effort为low。超时120秒，额外重试0次；服务失败不返回预设答案。

nanobot 0.3.5的SkillsLoader和受限ExecTool已完成真实加载执行。另通过完整nanobot agent自然语言入口实测：Agent读取Skill、生成输入文件、调用脚本，将摘要任务分配给Pro，并返回答案和新增调用数。临时输入文件已清理。

编排启动脚本只通过子进程环境变量传递已保存密钥。nanobot要求会话位于工作区之外，公开配置和会话因此保存在用户应用数据目录。任务模型和编排模型分别计费，网页统计只覆盖任务模型。

3 路由策略与统一接口

先识别代码和复杂推理，再识别摘要。难度分由推理关键词最多3分、类型最多2分、长度最多2分组成。命中代码或推理特征，或总分至少4，选推理档；摘要或输入估算超过600 Token选均衡档；其余选快速档。质量优先提升一档，成本优先允许短摘要降档，复杂任务保留能力下限。

输入估算采用中文字符乘1.5、其他字符约每3个计1 Token，再加系统提示与消息开销。这是选型启发式，不是精确分词器。项目保守上下文限额65536 Token；不足时升级或拒绝，不静默截断材料。

档位	真实模型	模式
eco	deepseek-flash	关闭思考
balanced	deepseek-v4-pro	关闭思考
reasoner	deepseek-v4-pro	开启思考

三个档位对应两个模型，均衡与推理是同一Pro模型的参数差异。输出预算包含推理Token；系统只展示最终答案，不保存思考正文。finish_reason为length时，标记incomplete并保留已知用量，提示增加预算；不完整回答不缓存。

CLI动作有plan、run、stats、doctor。stdin为UTF-8单个JSON，stdout返回单个JSON；退出码0成功、1业务错误、2参数错误。prompt最多24000字符，preference为balanced/cost/quality，max_tokens为64 - 8192、默认2048，use_cache默认true。mode兼容字段只接受live。

run要求confirm_live=true，网页执行按钮前说明文本外发及计费；plan只返回档位、依据和预算，不调用模型、不需要Key。缺少材料、空输入、未知字段和越界预算在调用前拒绝。Skill提供多条自然语言及JSON示例，可整体移植到其他工作区。

4 测试方法与真实调用结果

环境为Windows、Python 3.11.9、LiteLLM 1.101.0、nanobot-ai 0.3.5、pytest 8.4.2。37项自动测试全部通过，证据为artifacts/test-results.xml。另有十条人工验收案例，全部符合预期，覆盖正常、边界、异常；小型功能集合不代表普遍路由准确率。

测试组	主要验证内容	结果
路由与参数	中英文类型、偏好、长文、上下文、错误码	通过
缓存和计量	过期容量、身份隔离、供应商缓存计价	通过
失败处理	未知usage、超时、不完整回答不缓存	通过
动态配置	加密保存、读新Key、不回显、固定端点	通过
HTTP和CLI	同源限制、JSON和退出码、静态文件限制	通过
Runtime	Skill、ExecTool、自然语言Agent真实执行	通过

真实评估使用问答、会议摘要和二分查找三条公开任务。首轮响应均非空、finish_reason=stop、模型ID正确、usage_source为provider_reported。以下是独立评估的单次观测，耗时包含网络影响。

任务	输入/输出Token	模型耗时ms
简单问答 / Flash	34 / 26	2409.98
会议摘要 / Pro	53 / 29	1646.26
代码推理 / Pro	48 / 278	3950.38

自动测试使用隔离替身验证确定性边界，不消耗API；真实评估另行授权。模型质量、限流与账单未作全面评测。完整Agent摘要实测新增一次任务模型调用，36个输入和6个输出Token；编排成本未计入本表。

5 低Token与成本优化

规则路由不调用额外分类模型。plan只分析本地文本，发给生成模型的内容仅为当前任务与简短系统提示。这减少路由开销，但关键词规则对隐含语义的理解有限，需要独立标注任务集评价误路由。

精确请求缓存的键覆盖规范化文本、模型、思考配置、输出预算、公开配置、密钥摘要与版本。代码内部缩进保留。容量128条、TTL10分钟，仅完整成功结果可缓存；更换密钥清空并隔离旧缓存。密钥摘要只在内存中使用。

实验先运行三条公开任务，再在同一服务对象中重复一轮。首次实际调用三次，后续全部命中本地缓存。没有另做“关闭缓存六次”的付费基线，也不把推算结果当观测值。

指标	首轮3个请求	重复3个请求
实际API调用	3	0
新增输入Token	135	0
新增输出Token	333	0
估算费用USD	0.00139044	0

六次请求中，三次直接复用答案，相比每请求调用一次的逻辑基线减少三次调用。结论只适用于完全重复任务，不能推广为任意任务费用下降50%。不同CLI进程不共享缓存；持续运行网页服务可展示命中效果，时效性任务可关闭缓存。

费用采用服务usage。供应商输入缓存命中与未命中分别按公开费率计价，输出包含推理Token；按请求开始的UTC时段选峰谷费率，配置核对日期2026-09-17。费用是估算，跨时段和价格变更可能带来差异，以账单为准。

本地缓存命中新Token、费用和调用数为0，original_usage保留首轮数据。缺少usage时费用为null，失败也不认定免费。供应商输入缓存分项缺失时采用未命中单价上界估算，结果标明来源。

6 安全、复现与局限

密钥经页面密码框或交互式终端输入，Windows以DPAPI绑定当前用户加密，其他系统以权限0600保存。每次调用读取配置，文件原子替换。GET设置接口不返回原值或尾号，端点固定为官方HTTPS域名，任务JSON不能指定地址、路径或凭据。

HTTP只监听127.0.0.1，验证Host和Origin，静态文件映射固定。答案通过textContent展示，生成代码不会执行。审计仅存哈希、档位、状态、用量、费用和耗时；缓存正文只在内存。哈希不等于加密；nanobot和供应商有各自日志策略。

复现时双击start-demo.bat，配置并测试连接，执行问答、摘要、代码，再重复观察缓存。pytest复现自动检查，evaluate.py --live复现三次真实生成与缓存，verify_nanobot.py验证工具接口，run_nanobot.py验证自然语言链路。详细命令在验收指南。

最终视频为真实浏览器操作，34.76秒、1440×1080、25fps、中文字幕，展示连接、三档任务和缓存。没有录入或显示密钥。视频和独立评估是不同批次，回答与Token可能不同；对应JSON分别保存。

限制包括启发式误判、Token预估误差、价格更新、网络延迟、缓存时效性与单进程串行处理。本地服务面向单用户验收，没有公网认证、任务队列或分布式缓存。后续可增加标注任务集、精确分词与并发限额。

7 参考资料与提交物

课程依据为任务书第4-7页；技术来源核对日期2026-09-17。开源许可原文位于licenses。

1. LiteLLM官方仓库：<https://github.com/BerriAI/litellm>
2. nanobot官方仓库：<https://github.com/HKUDS/nanobot>
3. DeepSeek模型与价格：https://api-docs.deepseek.com/quick_start/pricing/
4. DeepSeek思考模式：https://api-docs.deepseek.com/guides/thinking_mode/

提交完整源码、README、Skill与CLI、依赖锁文件、十条案例、测试与实测证据、实验报告PDF及Markdown、现场指南和视频。个人信息在student-info.json填写；提交包排除密钥、运行日志和虚拟环境。